

Perguntas Teóricas Possíveis — Recurso BD 2025/2026

Premissa: O professor **NÃO repete** as perguntas da Época Normal no Recurso. Este ficheiro exclui todas as perguntas que saíram no Exame Teórico da Época Normal 2025/2026 e reúne todas as restantes perguntas da base de dados de exames do professor (de onde ele escolhe 6 perguntas teóricas para cada exame).

Perguntas JÁ ELIMINADAS (Saíram na Época Normal 2025/2026)

#	Tema	Pergunta EN 25/26
1	Definição de Termos	Defina: BD, SGBD (componentes), Metadados
2	LDD vs LMD	Diferenças entre LDD e LMD + operações
3	Vistas vs Relações Base	O que é uma vista + diferenças com relação base
4	Funções Agregação + NULLs	Restrições das funções de agregação + efeito dos NULLs
5	Mecanismo de Resolução de Vistas	Como funciona o mecanismo de resolução de vistas
6	Técnicas de Descoberta de Factos	Propósito e descrição de cada técnica

 **Nota:** A Pergunta 7 (Normalização de Fatura) e a Pergunta 8 (Modelação + SQL + Álgebra Relacional) saem **SEMPRE**, mas com enunciados diferentes (documento/fatura e cenário novos).

PERGUNTAS COM MAIOR PROBABILIDADE DE SAIR NO RECURSO

 **PRIORIDADE MÁXIMA (Frequência Altíssima + Não saíram na EN 25/26)**

P1 — Integridade Referencial + ON DELETE / ON UPDATE

Frequência: 8+ exames | **Aparece nos Modelos de Recurso:**  Modelo 1

Pergunta esperada:

Explique o que entende por Integridade Referencial. Identifique e descreva quais as ações que se podem utilizar nas subcláusulas ON DELETE e ON UPDATE.

Resposta Rápida: A **Integridade Referencial** é uma regra fundamental do modelo relacional que garante a consistência lógica entre tabelas relacionadas. Estabelece que se um registo numa tabela filha referencia um registo numa tabela pai (através de uma Chave Estrangeira - FK), o valor referenciado tem de existir como Chave Primária (PK) na tabela pai ou, em alternativa, ser nulo (NULL).

Para gerir a eliminação ou atualização de chaves primárias na tabela pai, o SQL disponibiliza as seguintes ações configuráveis nas cláusulas `ON DELETE` e `ON UPDATE`:

- **CASCADE:** As alterações propagam-se automaticamente. Apagar ou atualizar uma PK na tabela pai elimina ou atualiza correspondentemente todas as linhas filhas associadas.
- **SET NULL:** A chave estrangeira (FK) nas linhas filhas é alterada para `NULL` quando o registo pai é apagado/atualizado (requer que a coluna admita nulos).
- **SET DEFAULT:** A chave estrangeira (FK) nas linhas filhas é alterada para o valor padrão (Default) configurado para essa coluna.
- **RESTRICT:** Impede imediatamente a eliminação ou atualização do registo pai se existirem registos filhos associados.
- **NO ACTION:** Semelhante ao `RESTRICT`. Rejeita a operação no registo pai se houver registos filhos. A diferença técnica é que alguns SGBDs permitem adiar esta verificação para o final da transação (*deferred check*), enquanto o `RESTRICT` verifica e bloqueia de imediato.

🚩 P2 — Normalização: Objetivos e Impacto no Desempenho

Frequência: 8+ exames | Aparece nos Modelos de Recurso: ✅ Modelo 2

Pergunta esperada:

No contexto do modelo relacional, quais os objetivos da normalização de dados? De que forma o processo de normalização poderá afetar, posteriormente, o desempenho da implementação?

Resposta Rápida: A **Normalização** é uma técnica formal de desenho de bases de dados relacionais que visa:

1. **Minimizar a redundância** de dados para poupar espaço de armazenamento.
2. **Eliminar anomalias de atualização** (inserção, remoção e modificação).
3. **Garantir a consistência e integridade** lógica das relações.

O impacto da normalização no **desempenho** da aplicação é misto:

- **Operações de Leitura (Consultas/OLAP):** Prejudicado. A decomposição de uma tabela em várias tabelas menores obriga à realização de múltiplas operações de junção (`JOIN`), o que aumenta o custo computacional, a utilização de memória e o número de acessos ao disco (I/O).
- **Operações de Escrita (Inserções, Atualizações, Eliminações/OLTP):** Beneficiado. Como as tabelas são mais "estreitas" e a informação não está duplicada, as operações de escrita são muito mais rápidas, efetuadas num único local e envolvem menos bloqueios (*locks*) de tabelas e atualizações de índices.

Definições das Formas Normais:

- **Primeira Forma Normal (1FN):** Todos os atributos devem ser atômicos (valores indivisíveis, sem grupos repetidos ou arrays) e deve existir uma chave primária identificadora.

- **Segunda Forma Normal (2FN):** Está na 1FN e todos os atributos não chave dependem na totalidade da chave primária (elimina a dependência parcial, o que só se aplica a chaves primárias compostas).
- **Terceira Forma Normal (3FN):** Está na 2FN e nenhum atributo não chave depende de outro atributo não chave (elimina dependências transitivas).
- **Boyce-Codd Normal Form (BCNF):** Versão mais forte da 3FN. Uma tabela está na BCNF se, para qualquer dependência funcional $X \rightarrow Y$, X é uma superchave.

🚩 P3 — Anomalias de Atualização

Frequência: 6+ exames | **Aparece nos Modelos de Recurso:** ✅ Modelo 3

Pergunta esperada:

Descreva os tipos de anomalias de atualização que podem ocorrer numa relação com dados redundantes. Dê exemplos.

Resposta Rápida: Quando o esquema de uma base de dados não está devidamente normalizado, a redundância de dados dá origem a três tipos de anomalias de atualização:

- **Anomalia de Inserção:** Ocorre quando é impossível introduzir certas informações na base de dados sem que outra informação independente também seja inserida. *Exemplo:* Numa tabela que mistura alunos e disciplinas, não é possível registar uma nova disciplina antes de ter pelo menos um aluno inscrito nela (caso a chave primária envolva o ID do aluno).
- **Anomalia de Remoção (Eliminação):** Ocorre quando a eliminação de um registo resulta na perda involuntária de outros dados distintos e importantes que deveriam ser mantidos. *Exemplo:* Se eliminarmos o único aluno inscrito numa determinada disciplina, todos os dados sobre essa disciplina (como o nome do professor e créditos) são perdidos para sempre.
- **Anomalia de Modificação (Alteração):** Ocorre quando a atualização de um dado duplicado exige que se alterem múltiplos registos para evitar inconsistências. Se nem todas as cópias forem alteradas, os dados ficam num estado inconsistente. *Exemplo:* Se o nome de um professor estiver guardado na linha de cada aluno, mudar o nome do professor exige alterar 100 registos. Se falhar um, o mesmo professor aparecerá com dois nomes diferentes.

🚩 P4 — Triggers de Bases de Dados

Frequência: 4+ exames | **Aparece nos Modelos de Recurso:** ✅ Modelo 1

Pergunta esperada:

O que são triggers e para que servem? Quais as vantagens e desvantagens? Identifique os tipos quanto ao momento de execução.

Resposta Rápida: Um **Trigger (Gatilho)** é um bloco de código procedural (armazenado na base de dados) que é executado (disparado) de forma automática pelo SGBD em resposta a eventos específicos como operações DML (`INSERT` , `UPDATE` , `DELETE`) numa tabela ou vista.

- **Propósitos:** Imposição de regras de negócio complexas que superam as restrições declarativas padrão (`CHECK`), automatização de auditorias (logs de alterações), manutenção de dados derivados/calculados e sincronização de tabelas relacionadas.
- **Vantagens:**
 - **Centralização:** A lógica de validação fica na BD, garantindo que qualquer aplicação (web, mobile, etc.) respeite as mesmas regras.
 - **Segurança e Auditoria:** Garante o registo de alterações de forma inviolável.
 - **Automatização:** Reduz o código repetitivo no lado da aplicação.
- **Desvantagens:**
 - **Opacidade (Dificuldade de Depuração):** A execução é implícita (invisível para a aplicação), tornando difícil rastrear erros e fluxos complexos.
 - **Desempenho:** Podem degradar a performance das escritas se executarem lógica pesada em cada linha alterada.
 - **Portabilidade:** A sintaxe dos triggers varia drasticamente entre SGBDs (MySQL, Oracle, SQL Server).
- **Tipos quanto ao Momento de Execução:**
 - **BEFORE:** Executa antes da operação DML ser gravada no disco (ideal para validação e modificação de dados de entrada).
 - **AFTER:** Executa após a operação DML ter sido validada e aplicada (ideal para auditoria e propagação de alterações).
 - **INSTEAD OF:** Executa em substituição da ação DML original (frequentemente usado em vistas não atualizáveis para direcionar os dados para as tabelas base).
- **Granularidade:** Podem ser de linha (`FOR EACH ROW` - corre para cada tuplo afetado) ou de instrução (`FOR EACH STATEMENT` - corre uma única vez por comando SQL).

🔥 🔥 **PRIORIDADE ALTA (Frequência Moderada-Alta + Não saíram na EN 25/26)**

🚩 **P5 — Sistemas de BD vs Ficheiros + Vantagens/Desvantagens SGBD**

Frequência: 7+ exames | **Aparece nos Modelos de Recurso:** ✅ Modelo 3

Pergunta esperada:

Descreva as principais características de um Sistema BD e compare com os Sistemas de Ficheiros. Enuncie as vantagens e desvantagens de um SGBD.

Resposta Rápida: Os **Sistemas de Ficheiros** tradicionais dispersam os dados por ficheiros independentes e o acesso é feito por programas específicos, gerando acoplamento entre a estrutura física e o código da aplicação. Os **Sistemas de BD** centralizam os dados com acesso intermediado pelo SGBD, mantendo um catálogo/dicionário que descreve os dados (metadados).

Diferenças Principais:

1. **Dependência de Dados:** Nos ficheiros, qualquer alteração na estrutura física exige reescrever o código da aplicação. Nos sistemas de BD, existe **Independência de Dados**.
2. **Redundância:** Nos ficheiros há redundância descontrolada. No SGBD, a redundância é controlada e centralizada.
3. **Concorrência e Transações:** SGBD possui mecanismos robustos para gerir acessos simultâneos sem corrupção (através do controlo de concorrência e propriedades ACID), enquanto os ficheiros oferecem suporte muito limitado.

Vantagens do SGBD:

- Controlo e redução da redundância de dados.
- Partilha concorrente e consistente de informação.
- Aplicação uniforme de restrições de integridade e segurança de acessos.
- Independência física e lógica dos dados.
- Mecanismos automáticos de cópia de segurança (backup) e recuperação de desastres.

Desvantagens do SGBD:

- **Complexidade:** Exige conhecimento especializado para instalação, configuração e administração (DBA).
- **Custo Inicial:** Elevados custos de licenças de software, hardware robusto e formação de pessoal.
- **Impacto da Falha:** A centralização torna o SGBD num ponto único de falha (*Single Point of Failure*) — se falhar, todas as aplicações param.

Quando utilizar Sistemas de Ficheiros: Em aplicações muito simples, de utilizador único, com baixos volumes de dados, ou onde os recursos de hardware (memória e CPU) sejam extremamente limitados (ex: sistemas embebidos simples).

🚀 P6 — Arquitetura ANSI/SPARC (Nível Conceptual)

Frequência: 5+ exames | **Aparece nos Modelos de Recurso:** ✅ Modelo 2

Pergunta esperada:

A arquitetura ANSI/SPARC identifica três níveis nos SGBD. Descreva pormenorizadamente o nível intermédio.

Resposta Rápida: O nível intermédio da arquitetura ANSI/SPARC é o **Nível Conceptual**.

- **Descrição:** Representa a visão lógica global de toda a base de dados, unificando a perspetiva de todos os utilizadores. É a descrição estruturada dos dados guardados sem qualquer detalhe de armazenamento físico.
- **Conteúdo:** Define as entidades, atributos, relacionamentos, tipos de dados das colunas, restrições de integridade (como chaves primárias, estrangeiras e restrições `CHECK`) e regras de segurança de acesso.
- **Objetivo:** Funciona como uma camada de prevenção que liberta as aplicações do conhecimento do armazenamento físico. É neste nível que se define o modelo de dados lógico (ex: modelo relacional).

Esquemas da Arquitetura ANSI/SPARC (Contexto):

- **Esquema Externo (Nível Externo/Visão):** Define as vistas personalizadas para os utilizadores/aplicações (cada um vê apenas os dados relevantes para si).
- **Esquema Conceptual (Nível Conceptual):** Estrutura lógica unificada e global descrita acima.

- **Esquema Interno (Nível Interno/Físico):** Define como os dados são realmente gravados no disco, incluindo caminhos de ficheiros, tamanhos de blocos, índices e técnicas de compressão.

📌 P7 — Independência de Dados

Frequência: 5+ exames | **Aparece nos Modelos de Recurso:** ☒ Modelo 1

Pergunta esperada:

Descreva o conceito de independência de dados e a sua importância. Diferencie entre independência física e lógica.

Resposta Rápida: A **Independência de Dados** é a capacidade de alterar o esquema de uma base de dados num determinado nível da arquitetura ANSI/SPARC sem a necessidade de reescrever os esquemas dos níveis superiores ou o código das aplicações. A sua importância reside na redução drástica dos custos de manutenção e na flexibilidade para evoluir o sistema.

Existem dois tipos:

- **Independência Física de Dados:** Refere-se à capacidade de modificar o esquema físico (interno) sem alterar o esquema conceptual ou lógico. *Exemplo:* Mudar a BD para outro disco físico, alterar a estrutura de ficheiros, ou criar/remover índices para melhorar a performance de consultas, sem que as tabelas ou aplicações tenham de ser reescritas.
- **Independência Lógica de Dados:** Refere-se à capacidade de alterar o esquema conceptual sem que os esquemas externos (vistas de utilizadores) ou as aplicações precisem de ser modificados. *Exemplo:* Adicionar uma nova tabela ou coluna, ou dividir uma tabela existente em duas (para normalização), mantendo o funcionamento das consultas antigas através da criação de vistas que simulam a estrutura anterior.

📌 P8 — Data Warehouses: Benefícios e Problemas

Frequência: 4+ exames | **Aparece nos Modelos de Recurso:** ☒ Modelo 1

Pergunta esperada:

Descreva os benefícios e problemas dos Data Warehouses. Distinga entre Data Warehouse e Data Mart.

Resposta Rápida: Um **Data Warehouse (DW)** é um repositório de dados histórico, integrado, orientado por temas e não-volátil, especificamente concebido para apoiar os processos de tomada de decisão e análise estatística (OLAP).

- **Benefícios:**
 - **Integração:** Consolida dados provenientes de fontes heterogéneas (diferentes BDs operacionais, folhas de cálculo) sob uma estrutura uniforme.
 - **Perspetiva Histórica:** Mantém dados ao longo de anos para análise de tendências, ao contrário das BDs transacionais que focam no presente.
 - **Performance:** Separa as consultas analíticas pesadas (OLAP) das bases de dados de produção (OLTP), evitando a lentidão destas últimas.

- **Problemas:**

- **Custo e Complexidade:** Projetos caros, demorados e com elevado risco de desvio de requisitos.
- **Processo ETL:** Extrair, Transformar e Carregar os dados das fontes para o DW é extremamente complexo devido a problemas de qualidade de dados.
- **Manutenção:** Requer atualizações contínuas sempre que as fontes de dados operacionais sofrem alterações de estrutura.

Diferença entre Data Warehouse e Data Mart:

- **Data Warehouse:** Repositório global que abrange os dados de toda a organização (visão corporativa completa).
- **Data Mart:** Um subconjunto do Data Warehouse focado exclusivamente num departamento ou área de negócio específica.
- **Razões para o Desenvolvimento de um Data Mart (em vez de um DW global):**
 - **Foco e Relevância:** Disponibilizar aos utilizadores de um determinado departamento acesso rápido e exclusivo apenas aos dados que lhes interessam.
 - **Performance:** Como o volume de dados é muito inferior, as consultas analíticas são executadas mais rapidamente.
 - **Estrutura Adequada:** Os dados são previamente moldados e estruturados de acordo com as necessidades específicas daquele departamento.
 - **Simplicidade de Construção:** Projetar e implementar um Data Mart é muito menos complexo do que construir um Data Warehouse corporativo unificado.
 - **Custo Reduzido:** O investimento financeiro necessário para criar e manter um Data Mart é significativamente inferior.

🔴 P25 — Definição de Chaves (Candidata, Primária, Estrangeira)

Frequência: 5+ exames | **Foco Recurso:** ☒ Essencial

Pergunta esperada:

Enuncie a definição de cada um dos seguintes termos: Chave Candidata, Chave Primária, Chave Estrangeira.

Resposta Rápida:

- **Chave Candidata:** É um conjunto mínimo de atributos (uma ou mais colunas) cujos valores identificam de forma unívoca cada tuplo numa relação. Deve respeitar duas propriedades: *Unicidade* (identifica unicamente a linha) e *Irreduzibilidade* (nenhum subconjunto desses atributos consegue manter a unicidade).
- **Chave Primária (Primary Key - PK):** É a chave candidata escolhida pelo desenhador da base de dados para ser o identificador principal e oficial dos tuplos da relação. Não pode conter valores nulos (`NULL`) e é usada para criar chaves estrangeiras noutras tabelas.
- **Chave Estrangeira (Foreign Key - FK):** É um atributo (ou conjunto de atributos) numa relação filha que referencia a chave primária (ou uma chave candidata) de uma relação pai, estabelecendo o relacionamento lógico e a restrição de integridade referencial entre as duas tabelas.

🔴 P26 — Duas Principais Regras de Integridade no Modelo Relacional

Frequência: 5+ exames | Foco Recurso: ✅ Essencial

Pergunta esperada:

Defina as duas principais regras de integridade no Modelo Relacional e explique o seu propósito.

Resposta Rápida: As duas regras fundamentais de integridade garantem a consistência lógica dos dados e o correto relacionamento entre tabelas:

1. Regra de Integridade da Entidade:

- *Definição:* Nenhum atributo que faça parte da Chave Primária (PK) de uma relação base pode assumir o valor nulo (`NULL`).
- *Propósito:* A chave primária serve para identificar de forma unívoca cada tuplo na relação. Se permitisse nulos, perderíamos a capacidade de diferenciar e referenciar os registos individualmente.

2. Regra de Integridade Referencial:

- *Definição:* Se uma relação contiver uma Chave Estrangeira (FK) que referencia a Chave Primária (PK) de outra relação pai, cada valor da FK deve corresponder exatamente a um valor de PK existente na tabela pai, ou ser completamente nulo (`NULL`).
- *Propósito:* Garante que não existem referências "órfãs" (registos filhos que apontam para registos pais inexistentes), preservando a consistência lógica das ligações.

🔴 P27 — Cláusula GROUP BY vs WHERE vs HAVING

Frequência: 4+ exames | Foco Recurso: ✅ Essencial

Pergunta esperada:

Explique o funcionamento da cláusula GROUP BY. Qual a diferença fundamental entre a utilização das cláusulas WHERE e HAVING?

Resposta Rápida:

- **Funcionamento do GROUP BY:** Esta cláusula agrupa linhas que possuem os mesmos valores em colunas especificadas. É tipicamente usada para calcular métricas agregadas (somas, médias, contagens) para cada grupo.
Restrição: Qualquer coluna projetada no `SELECT` que não esteja dentro de uma função de agregação tem obrigatoriamente de constar na cláusula `GROUP BY`.
- **Diferença entre WHERE e HAVING:**
 - **WHERE :** É um filtro aplicado a **linhas individuais** *antes* de os dados serem agrupados pelo `GROUP BY`. Não pode conter funções de agregação (ex: `WHERE SUM(nota) > 10` é inválido).
 - **HAVING :** É um filtro aplicado a **grupos de linhas** *depois* de o `GROUP BY` ter sido executado. É especificamente concebido para filtrar com base em condições que envolvam funções de agregação (ex: `HAVING COUNT(*) > 5` é válido).

🔥 P28 — Circunstâncias para usar Desnormalização (com exemplo)

Frequência: 3+ exames | Foco Recurso: ✅ Essencial

Pergunta esperada:

Em que circunstâncias devemos recorrer à desnormalização de dados? Dê um exemplo prático.

Resposta Rápida: A **Desnormalização** é o processo intencional de introduzir redundância numa base de dados previamente normalizada para melhorar o desempenho operacional.

- **Circunstâncias para utilização:**

- Quando a performance de consultas de leitura (`SELECT`) é crítica e está degradada devido ao elevado custo de junções (`JOINS`) de múltiplas tabelas decompostas.
- Em sistemas onde a taxa de leitura é muitíssimo superior à taxa de escrita (ex: sistemas OLAP, Data Warehouses, catálogos públicos).
- Quando as tabelas envolvidas raramente sofrem atualizações (`UPDATE` / `DELETE`), o que minimiza o risco de anomalias e o overhead de manter a redundância sincronizada.

- **Exemplo Prático:** Numa tabela de `Post` e outra de `Comentario`, em vez de realizarmos um `COUNT` e `JOIN` em tempo real para mostrar a contagem de comentários de um post (operação efetuada milhões de vezes pelos leitores), adicionamos a coluna `TotalComentarios` diretamente na tabela `Post`. Esta coluna é atualizada apenas quando um comentário é inserido (baixa frequência), mas acelera imenso a leitura da página principal do blog.

🔥 PRIORIDADE MODERADA (Menos frequentes mas presentes nos Modelos de Recurso)

🔥 P9 — LMD Procedimentais vs Não-Procedimentais

Frequência: 2+ exames | Aparece nos Modelos de Recurso: ✅ Modelo 2

Pergunta esperada:

Explique as diferenças entre LMD procedimentais e não-procedimentais. Dê exemplos.

Resposta Rápida: As Linguagens de Manipulação de Dados (LMD) dividem-se em dois tipos de acordo com a forma de obtenção dos dados:

- **LMD Procedimentais:** O utilizador especifica **o que** quer e detalha passo a passo **como** obter os dados através de algoritmos. Funcionam tipicamente num registo de cada vez (*one-record-at-a-time*), necessitando de ciclos de repetição e cursores. *Exemplos:* Álgebra Relacional, linguagens procedimentais acopladas a SQL (PL/SQL na Oracle, T-SQL no SQL Server quando utilizam cursores e loops).
- **LMD Não-Procedimentais (Declarativas):** O utilizador apenas especifica **o que** quer obter, sem indicar os passos para a recolha física. O SGBD, através do seu otimizador de consultas, decide qual o melhor caminho de execução. Funcionam sobre conjuntos de registos de cada vez (*set-at-a-time*). *Exemplos:* SQL (especificamente comandos `SELECT`), Cálculo Relacional.

🚩 P10 — Subquery vs Junção

Frequência: 3+ exames | Aparece nos Modelos de Recurso: ✅ Modelo 1

Pergunta esperada:

Qual a diferença entre uma subquery e uma junção? Em que situações não é possível usar uma subquery?

Resposta Rápida:

- **Subquery (Subconsulta):** É uma consulta `SELECT` aninhada dentro de outra instrução SQL (como no `WHERE`, `HAVING`, `FROM` ou `SELECT` principal). É tipicamente utilizada para filtrar dados da consulta principal.
- **Junção (Join):** É uma operação que combina colunas de duas ou mais tabelas num único resultado consolidado, com base em colunas comuns (chaves primárias e estrangeiras).

Situações em que não é possível usar apenas uma subquery (necessitando de JOIN):

1. **Exibição de Atributos de Tabelas Distintas:** Quando o resultado final da consulta precisa de mostrar, no `SELECT` principal, colunas pertencentes a diferentes tabelas. Uma subquery colocada no `WHERE` apenas serve para filtrar a tabela principal, mas não consegue projetar as colunas da tabela interna na resposta final.
2. **Eficiência de Junções Múltiplas:** Embora teoricamente possível com subqueries correlacionadas pesadas, junções são a única via prática e otimizada quando se pretende relacionar muitas tabelas simultaneamente e expor múltiplos dados cruzados.

🚩 P11 — Arquitetura Cliente-Servidor (2 vs 3 Níveis)

Frequência: 3+ exames | Aparece nos Modelos de Recurso: ✅ Modelo 2

Pergunta esperada:

Compare a arquitetura cliente-servidor de 2 e 3 níveis. Qual a mais adequada para a Web?

Resposta Rápida:

- **Arquitetura de 2 Níveis (2-tier):** O cliente (geralmente um programa instalado - *fat client*) comunica diretamente com o servidor de bases de dados. A interface gráfica e a lógica de negócio (regras de decisão) correm no computador do cliente, enquanto o servidor apenas processa consultas SQL e armazena os dados.
- **Arquitetura de 3 Níveis (3-tier):** Introduce uma camada intermédia (Servidor de Aplicação/Web Server). O cliente é leve (*thin client* - ex: um browser). A lógica de negócio está centralizada no Servidor de Aplicação, que faz a ponte comunicando com a base de dados (Servidor de BD).

Mais adequada para a Web: A arquitetura de 3 níveis.

- **Vantagens na Web:**
 1. **Segurança:** O utilizador final não tem acesso direto nem credenciais da BD (esta fica protegida atrás da firewall da aplicação).
 2. **Escalabilidade:** Permite pooling de conexões à BD (partilha de ligações em vez de abrir uma por utilizador) e balanceamento de carga no servidor de aplicação.

3. **Manutenção:** Atualizações de lógica de negócio são feitas no servidor centralizado, sem necessidade de atualizar programas nos computadores dos clientes.

🚩 P12 — Atributos no Modelo Entidade-Relacionamento

Frequência: 3+ exames | **Aparece nos Modelos de Recurso:** ☒ Modelo 2

Pergunta esperada:

Descreva os atributos num diagrama ER. Dê exemplos de simples, compostos, multi-valor e derivados.

Resposta Rápida: Atributos são propriedades ou características que descrevem e qualificam uma Entidade ou um Relacionamento no modelo ER. Podem ser classificados como:

- **Simple (Atômicos):** Indivisíveis. Contêm apenas um valor unitário. *Exemplo:* NIF ou Sexo. Representados por uma elipse simples.
- **Compostos:** Podem ser divididos em sub-atributos menores e independentes. *Exemplo:* Morada (decomponível em Rua, Código Postal e Localidade). Representados por uma elipse ligada a outras elipses.
- **Multi-valorados (Multivalorados):** Podem conter um conjunto de valores para a mesma entidade. *Exemplo:* Telefone (uma pessoa pode ter vários números) ou Email. Representados por uma elipse com contorno duplo.
- **Derivados:** O seu valor não é armazenado diretamente, mas sim calculado a partir de outros atributos ou dados do sistema. *Exemplo:* Idade (calculada a partir da Data de Nascimento) ou TotalFatura. Representados por uma elipse tracejada.
- **Chave (Identificador):** Atributo cujos valores identificam de forma única cada ocorrência da entidade. Representado por texto sublinhado dentro da elipse.

🚩 P13 — Cursores SQL

Frequência: 2+ exames | **Aparece nos Modelos de Recurso:** ☒ Modelo 2

Pergunta esperada:

O que são cursores SQL? Qual o propósito? Descreva o ciclo de vida.

Resposta Rápida: Um **Cursor** é uma estrutura de controlo disponibilizada pelas extensões procedimentais do SQL (como PL/SQL e T-SQL) que funciona como um apontador para um conjunto de linhas devolvido por uma consulta.

- **Propósito:** Permitir o processamento de registos linha a linha, ao contrário do comportamento padrão do SQL que é orientado a conjuntos. É necessário quando a lógica requer cálculos ou validações específicas para cada tuplo individualmente.

Ciclo de Vida de um Cursor:

1. **DECLARE:** Define o nome do cursor e associa-o à consulta `SELECT` cujos registos pretende ler.
2. **OPEN:** Executa a consulta associada, aloca memória e posiciona o cursor antes da primeira linha.
3. **FETCH:** Recupera os valores da linha corrente para variáveis e avança o cursor para a linha seguinte. É usado dentro de um ciclo até que todas as linhas sejam processadas.

4. **CLOSE:** Fecha o cursor, libertando o conjunto de resultados ativos e quaisquer bloqueios de concorrência, mas mantém a sua estrutura definida na memória.
5. **DEALLOCATE (ou liberação):** Remove permanentemente a definição do cursor da memória e liberta todos os recursos do sistema.

🚩 P29 — Papel do Gestor de BD (Administrador de BD)

Frequência: 3+ exames | Foco Recurso: ☒ Médio

Pergunta esperada:

Descreva o papel de um gestor de BD (ou administrador de BD) numa solução baseada na mesma.

Resposta Rápida: Numa solução baseada em bases de dados, o **Administrador de Base de Dados (DBA)** desempenha um papel central e crítico, pois é o responsável por gerir e garantir a integridade, segurança, desempenho e disponibilidade dos dados da organização. Como todas as interações com a base de dados dependem do SGBD, o gestor de BD deve possuir um conhecimento profundo do SGBD para:

1. **Definição e Criação:** Definir o esquema conceptual e físico da base de dados.
2. **Segurança e Controlo de Acesso:** Criar contas de utilizadores, definir privilégios (GRANT/REVOKE) e garantir a proteção dos dados.
3. **Manutenção e Recuperação:** Definir e executar políticas de cópias de segurança (backups) e procedimentos de recuperação em caso de falha.
4. **Otimização de Desempenho (Tuning):** Monitorizar a performance das consultas, criar índices apropriados e ajustar configurações de hardware/software.

🚩 P30 — Funções que um SGBD deve satisfazer

Frequência: 4+ exames | Foco Recurso: ☒ Médio

Pergunta esperada:

Identifique as funções que um SGBD deve satisfazer para uma gestão eficaz da base de dados.

Resposta Rápida: Para garantir o correto funcionamento e integridade dos dados, um SGBD deve disponibilizar as seguintes 10 funções principais:

1. **Armazenamento, Pesquisa e Atualização:** Permitir aos utilizadores ler, inserir, atualizar e apagar dados na BD.
2. **Dicionário de Dados / Catálogo:** Manter um catálogo com a descrição dos dados (metadados) acessível aos utilizadores.
3. **Suporte a Transações:** Garantir que as alterações são feitas de forma atómica e consistente (princípio do tudo ou nada).
4. **Serviços de Controlo de Concorrência:** Assegurar que a BD pode ser acedida simultaneamente por vários utilizadores sem perda de consistência.

5. **Serviços de Recuperação (Recovery):** Mecanismos para recuperar a BD de falhas de hardware ou software (devolvendo-a a um estado consistente).
6. **Serviços de Autenticação e Autorização:** Garantir que apenas utilizadores autorizados acedem a dados específicos.
7. **Suporte a Comunicação de Dados:** Permitir a integração e acesso à BD em rede (ambientes distribuídos/cliente-servidor).
8. **Serviços de Integridade:** Impor regras de integridade declaradas no esquema (como chaves primárias, restrições CHECK, integridade referencial).
9. **Serviços para Independência de Dados:** Fornecer suporte para separar as aplicações da estrutura física e lógica dos dados.
10. **Utilitários:** Disponibilizar ferramentas de administração (monitorização, importação/exportação, tuning).

🚀 P31 — As Três Gerações de SGBD

Frequência: 3+ exames | Foco Recurso: ✅ Médio

Pergunta esperada:

Descreva brevemente as três gerações de SGBD, indicando os respetivos modelos de dados.

Resposta Rápida: A evolução dos SGBDs divide-se em três gerações tecnológicas:

1. **1ª Geração (Modelos Hierárquico e de Rede):** Desenvolvidos nos anos 60/70. Organizavam os dados em árvores (hierárquico) ou grafos (rede). Apresentavam baixa independência de dados (aplicações muito acopladas à estrutura física) e exigiam algoritmos de acesso complexos (navegação por ponteiros).
2. **2ª Geração (Modelo Relacional):** Desenvolvido nos anos 70/80 (proposto por E. F. Codd). Organiza os dados em tabelas (relações) bidimensionais. Resolveu a dependência física/lógica e simplificou o acesso através de linguagens declarativas (como SQL).
3. **3ª Geração (Modelos Orientado a Objetos e Objeto-Relacional):** Desenvolvidos a partir dos anos 90. Integram o paradigma da Programação Orientada a Objetos com a persistência de bases de dados, suportando dados complexos (multimédia, CAD/CAM, SIG) e herança.

🚀 P32 — Função de um System Catalog (Dicionário de Dados)

Frequência: 3+ exames | Foco Recurso: ✅ Médio

Pergunta esperada:

Discuta a função e a importância de um System Catalog no contexto de um SGBD.

Resposta Rápida: O **System Catalog** (ou Catálogo do Sistema/Dicionário de Dados) é uma das componentes mais cruciais de um SGBD. Funciona como um repositório central que armazena os **metadados** (dados que descrevem a estrutura e definição de toda a base de dados).

- **Funções principais:**

- Armazenar o esquema concetual (tabelas, colunas, tipos de dados, chaves primárias/estrangeiras e restrições CHECK).
 - Registrar o esquema físico (caminhos de ficheiros, partições e índices criados).
 - Guardar o esquema externo (definição de vistas/views).
 - Controlar a segurança (utilizadores autorizados, senhas, papéis e privilégios de acesso).
- **Importância:** Garante a **independência de dados**, pois as aplicações consultam o catálogo em tempo de execução para obter a descrição das tabelas em vez de terem de codificar essa estrutura diretamente no código-fonte.

✦ P33 — Propósito e uso de Índices sobre Relações

Frequência: 4+ exames | Foco Recurso: ☒ Médio

Pergunta esperada:

Descreva o propósito da criação de índices sobre relações e em que situações será preferível criá-los ou não.

Resposta Rápida:

- **Propósito:** Um índice é uma estrutura física auxiliar criada para **acelerar a velocidade de pesquisa e obtenção de registos**. Funciona de forma análoga ao índice remissivo de um livro: em vez de o SGBD ler a tabela inteira linha a linha (Full Table Scan), pesquisa primeiro o índice para obter a localização física (endereço em disco) do registo e acede-lhe diretamente.
- **Quando criar índices:**
 - Em colunas frequentemente utilizadas em cláusulas de filtragem (`WHERE`).
 - Em colunas usadas para realizar junções de tabelas (`JOIN`).
 - Em colunas de ordenação ou agrupamento (`ORDER BY` ou `GROUP BY`).
 - Em tabelas com grande volume de dados.
- **Quando NÃO criar índices:**
 - Em tabelas pequenas (onde a pesquisa sequencial é mais rápida).
 - Em colunas com baixa seletividade (ex: coluna "Sexo", onde os valores se repetem em quase toda a tabela).
 - Em tabelas que sofrem constantes operações de escrita (`INSERT` , `UPDATE` , `DELETE`), porque o SGBD é obrigado a atualizar o índice a cada modificação, degradando drasticamente o desempenho de escrita.

✦ P34 — Controlo de Concorrência e a sua Importância

Frequência: 3+ exames | Foco Recurso: ☒ Médio

Pergunta esperada:

O que é controlo de concorrência e porque é que é importante num SGBD?

Resposta Rápida: O **Controlo de Concorrência** é o mecanismo do SGBD responsável por gerir o acesso simultâneo de múltiplos utilizadores ou transações aos mesmos dados na base de dados.

- **Importância:** Sem um controlo de concorrência eficaz, transações simultâneas que escrevem nas mesmas tabelas podem corromper os dados, resultando em anomalias graves como:
 - *Atualização Perdida (Lost Update):* Uma transação sobrepõe e apaga a alteração feita por outra transação concorrente.
 - *Leitura Inconsistente (Unrepeatable Read):* Uma transação lê o mesmo registo duas vezes e obtém valores diferentes porque outra transação o alterou no meio.
 - *Leitura Suja (Dirty Read):* Uma transação lê dados que foram alterados por outra transação que ainda não fez COMMIT. O controlo de concorrência garante a propriedade de **Isolamento (ACID)** através de técnicas como Bloqueios (Locks/2PL) ou Selos Temporais (Timestamps).

🚩 P35 — Conceito de Database Schema e seus Três Níveis

Frequência: 4+ exames | Foco Recurso: ✅ Médio

Pergunta esperada:

Explique o conceito de Database Schema e descreva os três tipos de schema numa base de dados relacional.

Resposta Rápida: Um **Database Schema** (Esquema da Base de Dados) é a descrição estruturada e lógica da base de dados, representando o seu desenho (design) e regras, sem conter os dados reais em si. De acordo com a arquitetura ANSI/SPARC de 3 níveis, existem três tipos de esquemas na BD:

1. **Esquema Interno (Nível Físico):** Descreve como os dados são fisicamente gravados e estruturados nos discos (ficheiros, blocos, compressão e índices).
2. **Esquema Conceptual (Nível Lógico Global):** Descreve a estrutura lógica de toda a BD de forma unificada (entidades, relacionamentos, atributos, chaves e restrições).
3. **Esquema Externo (Nível de Visão):** Descreve as vistas específicas para utilizadores ou aplicações individuais (cada um interage apenas com os dados autorizados).

🚩 P36 — Objetivos da Arquitetura de Três Níveis ANSI/SPARC

Frequência: 3+ exames | Foco Recurso: ✅ Médio

Pergunta esperada:

Identifique quais os objetivos da arquitetura de três níveis ANSI/SPARC.

Resposta Rápida: O objetivo principal da arquitetura ANSI/SPARC de 3 níveis é a **separação da visão do utilizador da representação física da base de dados**. Isto traduz-se nos seguintes objetivos específicos:

1. **Vistas Personalizadas:** Permitir que múltiplos utilizadores acessem aos mesmos dados, mas com vistas personalizadas às suas necessidades.
2. **Isolamento de Alterações:** Garantir que um utilizador não possa interferir com as vistas de outros utilizadores.
3. **Independência Física:** Permitir alterar a estrutura física (discos, índices) sem afetar o esquema conceptual ou as aplicações.

4. **Independência Lógica:** Permitir alterar o esquema conceptual (adicionar/remover colunas ou tabelas) sem afetar os esquemas externos ou as aplicações.
5. **Independência do Hardware:** Garantir que a estrutura lógica seja independente das características físicas do hardware de rede e computadores.

🚩 P37 — Propriedades das Relações no Modelo Relacional

Frequência: 3+ exames | Foco Recurso: ✅ Médio

Pergunta esperada:

Identifique e descreva quais as propriedades das Relações no contexto do Modelo Relacional.

Resposta Rápida: Uma relação (que corresponde logicamente a uma tabela) deve cumprir as seguintes propriedades estruturais:

1. **Nome Único:** Cada relação no esquema tem de ter um nome distinto de todas as outras.
2. **Valores Atômicos (Atomicidade):** A interseção de qualquer linha e coluna deve conter exatamente um único valor indivisível (não são permitidos grupos repetidos ou arrays).
3. **Atributos Distintos:** Cada coluna (atributo) deve ter um nome único dentro daquela relação.
4. **Domínio Homogêneo:** Todos os valores numa determinada coluna devem pertencer ao mesmo domínio (tipo de dados).
5. **Ordem das Colunas Irrelevante:** A ordem física das colunas na relação não tem qualquer significado lógico.
6. **Tuplos Únicos (Sem Duplicados):** Não podem existir duas linhas (tuplos) idênticas na relação.
7. **Ordem das Linhas Irrelevante:** A ordem física em que as linhas são armazenadas não afeta o significado dos dados.

🚩 P38 — Cláusulas do Comando SELECT

Frequência: 4+ exames | Foco Recurso: ✅ Médio

Pergunta esperada:

Explique o significado e a função de cada uma das cláusulas utilizadas na estrutura de um comando SELECT.

Resposta Rápida: A estrutura padrão de uma consulta SQL baseia-se nas seguintes cláusulas ordenadas:

- **SELECT :** (Obrigatório) Especifica as colunas, expressões ou resultados de funções agregadas que devem constar no resultado.
- **FROM :** (Obrigatório) Identifica a tabela ou conjunto de tabelas (e junções **JOIN**) de onde vêm os dados.
- **WHERE :** (Opcional) Filtra os registos individuais da tabela base com base em condições lógicas (executado *antes* do agrupamento).
- **GROUP BY :** (Opcional) Agrupa os registos que partilham os mesmos valores, permitindo realizar cálculos agregados por grupo.

- **HAVING** : (Opcional) Filtra os grupos formados pela cláusula **GROUP BY** (executado *depois* do agrupamento e aceitando funções agregadas).
- **ORDER BY** : (Opcional) Ordena o conjunto de resultados final em ordem ascendente (**ASC**) ou descendente (**DESC**).

🚩 P39 — Importância do WHERE em UPDATE e DELETE

Frequência: 3+ exames | Foco Recurso: ✅ Médio

Pergunta esperada:

Explique a importância e a aplicação prática da cláusula WHERE em comandos UPDATE e DELETE.

Resposta Rápida: Nos comandos SQL de manipulação de dados **UPDATE** (alteração) e **DELETE** (eliminação), a cláusula **WHERE** funciona como um filtro de seleção que indica quais os registros específicos da tabela que devem sofrer a alteração ou remoção.

- **Importância Crítica:** Na ausência da cláusula **WHERE**, os comandos **UPDATE** e **DELETE** aplicam-se imediatamente a **todos os registros** presentes na tabela ativa.
 - *Exemplo sem WHERE:* **DELETE FROM Aluno;** elimina permanentemente todos os alunos da base de dados.
 - *Exemplo com WHERE:* **DELETE FROM Aluno WHERE IDJogador = 5;** elimina apenas o aluno com o identificador correspondente a 5, preservando os restantes registros da tabela.

🚩 P40 — Combinação de Resultados de duas Queries (Operações de Conjuntos)

Frequência: 3+ exames | Foco Recurso: ✅ Médio

Pergunta esperada:

Como poderão ser combinados os resultados de duas queries SQL? Identifique quais as condições necessárias.

Resposta Rápida: Os resultados de duas queries independentes podem ser combinados no SQL utilizando operadores baseados na teoria dos conjuntos:

1. **UNION** : Devolve a união de ambos os resultados (todas as linhas de ambas as queries, eliminando duplicados por padrão).
 2. **INTERSECT** : Devolve a interseção (apenas as linhas comuns a ambos os resultados).
 3. **EXCEPT (ou MINUS)** : Devolve a diferença (linhas que aparecem no resultado da primeira query mas não na segunda).
- **Condições Necessárias (Union Compatibility):** Para que o SGBD permita combinar duas queries, estas têm de ser compatíveis com a união. Isto significa que:
 - Devem projetar exatamente o **mesmo número de colunas/atributos**.
 - As colunas correspondentes (pela ordem em que são listadas no **SELECT**) devem possuir **domínios (tipos de dados) compatíveis**.

🚩 P41 — Os Três Tipos de Subqueries

Frequência: 3+ exames | Foco Recurso: ✅ Médio

Pergunta esperada:

Diferencie entre os 3 tipos de subqueries SQL com base no resultado que devolvem.

Resposta Rápida: Uma subquery (subconsulta) é um comando `SELECT` aninhado dentro de outra instrução SQL. Classificam-se em 3 tipos de acordo com a estrutura do resultado devolvido:

1. **Subquery Escalar (Scalar Subquery):** Devolve um valor único (uma linha e uma coluna). Pode ser utilizada na lista de projeção do `SELECT` ou no `WHERE` com operadores de comparação simples (ex: `=`, `>`, `<`). *Exemplo:* `WHERE Salario > (SELECT AVG(Salario) FROM Funcionario)`
2. **Subquery de Linha (Row Subquery):** Devolve um único tuplo composto por múltiplas colunas. Permite comparar múltiplos atributos de uma só vez. *Exemplo:* `WHERE (CodCurso, Ano) = (SELECT CodCurso, Ano FROM Cadeira WHERE ID = 3)`
3. **Subquery de Tabela (Table Subquery):** Devolve uma relação completa (múltiplas linhas e múltiplas colunas). É utilizada com operadores de conjunto (ex: `IN`, `EXISTS`, `ANY`, `ALL`) ou colocada diretamente na cláusula `FROM` como uma tabela derivada.

🚩 P42 — Mecanismos de Controlo de Acesso em SQL (GRANT/REVOKE)

Frequência: 3+ exames | Foco Recurso: ✅ Médio

Pergunta esperada:

Descreva como funcionam os mecanismos de controlo de acesso em SQL (segurança) e o papel dos privilégios.

Resposta Rápida: A segurança das bases de dados em SQL baseia-se em **identificadores de autorização (utilizadores)** e **privilégios**.

- **Identificador de Autorização:** Cada utilizador tem uma conta criada na BD.
- **Privilégios:** Direitos de executar ações específicas em objetos da BD (ex: ler, escrever, alterar). Podem ser privilégios de sistema (ex: criar tabelas) ou privilégios sobre objetos (ex: `SELECT`, `INSERT`, `UPDATE`, `DELETE` numa tabela).
- **Mecanismo de Controlo:**
 - **GRANT** : Concede privilégios a utilizadores. *Sintaxe:* `GRANT SELECT, UPDATE ON Aluno TO vasco;` (permite ao utilizador 'vasco' ler e atualizar a tabela Aluno).
 - **REVOKE** : Retira privilégios previamente atribuídos. *Sintaxe:* `REVOKE UPDATE ON Aluno FROM vasco;` (retira o direito de atualizar a tabela Aluno). Qualquer objeto tem um proprietário (owner) que detém todos os privilégios sobre o mesmo por defeito e pode distribuí-los ou revogá-los.

🚩 P43 — Vantagens e Desvantagens das Vistas

Frequência: 3+ exames | Foco Recurso: ✅ Médio

Pergunta esperada:

Discuta as vantagens e desvantagens da utilização de vistas (views) no contexto de bases de dados.

Resposta Rápida:

- **Vantagens:**
 - **Segurança Melhorada:** Permite expor apenas colunas não confidenciais de uma tabela, escondendo dados sensíveis dos utilizadores.
 - **Simplicidade de Consulta:** Esconde junções (`JOINS`) complexas e filtros pesados sob um nome de vista simples.
 - **Personalização:** Diferentes utilizadores podem ver os mesmos dados base estruturados de formas diferentes.
 - **Independência Lógica:** Se uma tabela base for alterada ou dividida, a vista pode ser recriada mantendo o funcionamento das aplicações sem alteração de código.
- **Desvantagens:**
 - **Desempenho:** A consulta subjacente é reexecutada a cada acesso (ao contrário das vistas materializadas).
 - **Restrições de Escrita:** Muitas vistas (especialmente as que contêm agregados, `DISTINCT` , `GROUP BY` ou junções) não admitem modificações diretas.
 - **Restrição Estrutural:** A estrutura da vista é rígida no momento da criação; qualquer alteração na tabela base exige a sua recriação.

★ PRIORIDADE SECUNDÁRIA (Menos frequentes, mas presentes em Modelos de Recurso específicos)

🚩 P14 — Materialização de Vistas (Indexed/Materialized Views)

Aparece nos Modelos de Recurso: ✅ Modelo 1

Pergunta esperada:

Explique o conceito de materialização de vistas. Vantagens e desvantagens vs vistas tradicionais? Em que contextos é recomendável?

Resposta Rápida: Ao contrário de uma vista tradicional (que é apenas uma consulta guardada em texto e reexecutada sempre que é chamada), uma **Vista Materializada** (ou Vista Indexada) tem o resultado da sua consulta fisicamente calculado e armazenado em disco, tal como se tratasse de uma tabela normal.

- **Vantagens:**
 - **Velocidade de Leitura:** Acelera drasticamente consultas complexas que envolvem agregações pesadas (`SUM` , `AVG`), ordenações ou junções de várias tabelas, pois o resultado já está pré-calculado.

- **Desvantagens:**
 - **Perda de Performance na Escrita:** Sempre que as tabelas base são modificadas, o SGBD precisa de recalculá-las e atualizar a vista materializada, gerando overhead.
 - **Armazenamento:** Consome espaço físico em disco.
- **Contextos Recomendáveis:**
 - Ambientes de **Data Warehouse e Business Intelligence (OLAP)** com grandes volumes de dados.
 - Tabelas de dados com elevada taxa de leitura e taxas de atualização/escrita muito baixas.

🚩 P15 — Operações de Junção (Theta, Equi, Natural, Outer, Semi)

Aparece nos Modelos de Recurso: ☒ Modelo 4

Pergunta esperada:

Descreva as diferenças entre Theta Join, Equijoin, Natural Join, Outer Join e Semijoin. Dê exemplos para suportar a sua resposta.

Resposta Rápida:

- **Theta Join ($R \bowtie_{\theta} S$):** Operação geral de junção que combina linhas de duas relações com base numa condição contendo qualquer operador de comparação ($=, >, <, \geq, \leq, \neq$).
- **Equijoin:** Caso particular do Theta Join onde a condição de junção utiliza exclusivamente o operador de igualdade ($=$). Mantém colunas com o mesmo nome duplicadas no resultado.
- **Natural Join ($R \bowtie S$):** Equijoin automático que compara colunas com o mesmo nome em ambas as tabelas. Ao contrário do Equijoin, remove colunas duplicadas do resultado final automaticamente.
- **Outer Join (Junção Externa - Esquerda, Direita ou Completa):** Devolve todos os registos correspondentes (Inner Join) e, adicionalmente, preserva os registos de uma ou de ambas as tabelas que não encontraram correspondência, preenchendo as colunas em falta com `NULL`.
- **Semijoin ($R \ltimes S$):** Devolve apenas as linhas da primeira tabela (R) que encontram correspondência na segunda tabela (S), mas sem expor atributos da segunda tabela nem duplicar linhas da primeira. Equivale a filtrar com `EXISTS`.

Exemplos Práticos: Dadas as tabelas:

- **ESTUDANTE** (ID, Nome, Turma) com os registos:
 - (1, 'Carlos', 111)
 - (2, 'Teixeira', 222)
 - (3, 'Alberto', 333)
- **CADEIRA** (Turma, NomeCadeira) com os registos:
 - (111, 'Base de Dados')
 - (222, 'Física')
- **Theta Join / Equijoin (com igualdade):** `ESTUDANTE \bowtie_{(Estudante.Turma = Cadeira.Turma)} CADEIRA` Resulta em: (1, 'Carlos', 111, 111, 'Base de Dados') e (2, 'Teixeira', 222, 222, 'Física') (a coluna `Turma` aparece

duplicada).

- **Natural Join:** `ESTUDANTE ⋈ CADEIRA` Resulta em: `(Turma, ID, Nome, NomeCadeira) → (111, 1, 'Carlos', 'Base de Dados')` e `(222, 2, 'Teixeira', 'Física')` (colunas homónimas são fundidas automaticamente).
- **Outer Join (Left):** `ESTUDANTE ⋈⋈ CADEIRA` Preserva registos do lado esquerdo. Além dos dois correspondentes, devolve o aluno 'Alberto': `(3, 'Alberto', 333, NULL, NULL)`.
- **Semijoin (R ⋈ S):** `ESTUDANTE ⋈ CADEIRA` Devolve apenas os atributos de `ESTUDANTE` para os alunos com correspondência em `CADEIRA`. Devolve exatamente as linhas correspondentes de `ESTUDANTE` (IDs 1 e 2) mas sem quaisquer atributos de `CADEIRA`.

🚩 P16 — Stored Procedures vs Funções (UDF)

Aparece nos Modelos de Recurso: ☒ Modelo 4

Pergunta esperada:

Diferença entre Stored Procedure e User-Defined Function? Três diferenças fundamentais.

Resposta Rápida: Embora ambos sejam blocos de código guardados no SGBD, distinguem-se pelas seguintes diferenças fundamentais:

Característica	Funções (User-Defined Functions - UDF)	Stored Procedures (Procedimentos)
Retorno de Valores	Obrigatório: Devolve um único valor ou uma tabela (<code>RETURN</code>).	Opcional: Pode não devolver nada ou usar parâmetros de saída (<code>OUT</code>).
Invocação	Integradas em comandos SQL (ex: <code>SELECT col, func() FROM tab</code>).	Invocadas isoladamente com comandos próprios (ex: <code>EXEC</code> ou <code>CALL</code>).
Modificação de Dados	Apenas Leitura: Não podem alterar dados nem tabelas (DML bloqueado).	Escrita: Podem efetuar <code>INSERT</code> , <code>UPDATE</code> e <code>DELETE</code> .
Transações	Não é permitido usar <code>COMMIT</code> ou <code>ROLLBACK</code> .	Permitem o controlo e gestão total de transações.

🚩 P17 — Sublinguagens de Dados (DDL, DML, DCL, TCL)

Aparece nos Modelos de Recurso: ☒ Modelo 3

Pergunta esperada:

O que são sublinguagens de dados? Identifique e descreva DDL, DML, DCL, TCL com exemplos.

Resposta Rápida: As sublinguagens de dados são divisões funcionais da linguagem SQL, agrupando comandos de acordo com o tipo de operação que realizam na base de dados:

- **DDL (Data Definition Language - Definição):** Comandos que definem, alteram ou eliminam as estruturas (esquemas) da base de dados. *Exemplos:* `CREATE` (tabelas, vistas, índices), `ALTER`, `DROP`, `TRUNCATE`.
- **DML (Data Manipulation Language - Manipulação):** Comandos que permitem interagir com os dados armazenados nas tabelas. *Exemplos:* `SELECT` (consulta), `INSERT` (inserção), `UPDATE` (alteração), `DELETE` (remoção).
- **DCL (Data Control Language - Controle):** Comandos para gestão de acessos, segurança e privilégios dos utilizadores. *Exemplos:* `GRANT` (atribuir privilégios), `REVOKE` (retirar privilégios).
- **TCL (Transaction Control Language - Transações):** Comandos para gerir os limites e estados das transações. *Exemplos:* `COMMIT` (gravar alterações), `ROLLBACK` (cancelar alterações), `SAVEPOINT` (ponto intermédio).

🚀 P18 — 5 Operações Básicas da Álgebra Relacional

Aparece nos Modelos de Recurso: ☒ Modelo 3

Pergunta esperada:

Defina as cinco operações básicas da Álgebra Relacional. Demonstre como Junção, Interseção e Divisão são derivadas.

Resposta Rápida: As cinco operações básicas e independentes que definem a álgebra relacional são:

1. **Seleção (σ):** Operação unária que filtra as linhas (tuplos) que satisfazem uma determinada condição.
2. **Projeção (π):** Operação unária que seleciona colunas (atributos) específicas de uma relação, eliminando linhas duplicadas.
3. **Produto Cartesiano ($\bowtie$):** Operação binária que combina todas as linhas da primeira relação com todas as linhas da segunda.
4. **União ($\cup$):** Operação binária que combina todas as linhas de duas tabelas (requer que as tabelas sejam compatíveis em número e tipo de colunas).
5. **Diferença ($-$):** Operação binária que devolve as linhas que estão na primeira relação mas não na segunda (requer compatibilidade).

Operações Derivadas (com demonstração matemática):

- **Junção (Join - $\bowtie$):** Combinação filtrada de duas tabelas. É derivada do Produto Cartesiano seguido de uma Seleção.

$$R \bowtie_{\text{condição}} S \equiv \sigma_{\text{condição}}(R \bowtie S)$$

- **Interseção ($\cap$):** Devolve as linhas comuns a duas tabelas. É derivada usando a operação de Diferença.

$$R \cap S \equiv R - (R - S)$$

- **Divisão ($R \div S$):** Devolve todos os valores de um conjunto de atributos de R que estão associados a todos os valores da relação S. *Demonstração:* Seja $R(X, Y)$ e $S(Y)$ onde X e Y são conjuntos de atributos. A divisão de R por S é definida como:

$$R \div S \equiv \pi_X(R) - \pi_X(\pi_X(R) \bowtie S - R)$$

🚩 P19 — Componentes do Ambiente de um SGBD

Aparece nos Modelos de Recurso: ☒ Modelo 4

Pergunta esperada:

Descreva os 5 componentes principais do ambiente de um SGBD.

Resposta Rápida: O funcionamento global de um SGBD baseia-se na interação de cinco componentes principais:

1. **Hardware:** A infraestrutura física que suporta o sistema, incluindo o servidor (CPU, memória RAM), dispositivos de armazenamento físico (discos SSD/HDD) e equipamentos de rede.
2. **Software:** O conjunto de programas, composto pelo próprio software do SGBD (motor da base de dados), o Sistema Operativo onde este corre e as aplicações cliente que acedem aos dados.
3. **Dados:** O componente mais importante. Engloba tanto os dados operacionais dos utilizadores como os metadados (dicionário de dados/catálogo que descreve a estrutura física e lógica da BD).
4. **Procedimentos:** As regras administrativas, instruções e políticas de funcionamento da base de dados (ex: backups, formas de iniciar sessão e recuperação de falhas).
5. **Utilizadores:** As pessoas envolvidas no sistema: Administradores de BD (DBAs), Projetistas/Designers de bases de dados, Programadores de aplicações e os Utilizadores Finais.

🚩 P20 — Conceitos do Modelo Relacional

Aparece nos Modelos de Recurso: ☒ Modelo 4

Pergunta esperada:

Explique: Relação, Atributo, Domínio, Tuplo, Grau e Cardinalidade.

Resposta Rápida:

- **Relação:** Uma tabela bidimensional de dados que contém linhas e colunas. Matematicamente, representa um subconjunto do produto cartesiano dos domínios dos seus atributos.
- **Atributo:** Uma coluna nomeada numa relação. Representa uma propriedade ou característica dos dados.
- **Domínio:** O conjunto de todos os valores atómicos e válidos permitidos para um determinado atributo.
- **Tuplo:** Uma linha na relação. Representa um registo individual completo (uma ocorrência de dados).
- **Grau:** O número total de atributos (colunas) que constituem a estrutura de uma relação.
- **Cardinalidade:** O número total de tuplos (linhas) presentes numa relação num determinado momento.

🚩 P21 — Vistas Atualizáveis

Aparece nos Modelos de Recurso: ☒ Modelo 4

Pergunta esperada:

Quais as restrições para uma vista ser atualizável diretamente via DML?

Resposta Rápida: Para que o SGBD consiga propagar sem ambiguidade operações DML (`INSERT` , `UPDATE` , `DELETE`) executadas sobre uma vista diretamente para as respetivas tabelas base, a vista tem de cumprir restrições restritas:

- **Tabela Base Única:** Deve mapear exatamente **uma única tabela base** no seu `FROM` (sem junções `JOIN`).
- **Sem Agregações:** A cláusula `SELECT` não pode conter funções de agregação (`SUM` , `AVG` , `COUNT` , `MIN` , `MAX`).
- **Sem Agrupamento ou Filtros de Grupo:** Não pode conter as cláusulas `GROUP BY` ou `HAVING` .
- **Sem Eliminação de Duplicados:** Não pode utilizar a palavra-chave `DISTINCT` .
- **Sem Operações de Conjuntos:** Não pode utilizar `UNION` , `INTERSECT` ou `EXCEPT` .
- **Atributos Obrigatórios:** Para inserções (`INSERT`), a vista deve incluir todas as colunas definidas como `NOT NULL` e sem valor por omissão (`DEFAULT`) na tabela base.
- **Sem Colunas Calculadas:** Não pode conter expressões ou atributos derivados na lista de seleção.

🔴 P22 — Especialização vs Generalização (Modelo ER)

Aparece nos Modelos de Recurso: ☒ Modelo 4

Pergunta esperada:

Diferenças entre especialização e generalização no diagrama ER. Dê exemplos.

Resposta Rápida: Ambos os conceitos gerem a hierarquia de herança no modelo ER, distinguindo-se pelo sentido do processo de modelação:

- **Especialização (Abordagem Top-Down / De cima para baixo):** Processo que consiste em identificar sub-entidades (subclasses) específicas a partir de uma entidade global (superclasse), com base em características ou funções distintas. *Exemplo:* A partir da entidade `Funcionário` , especializam-se as subclasses `Programador` e `Gestor` .
- **Generalização (Abordagem Bottom-Up / De baixo para cima):** Processo inverso que consiste em identificar características comuns em várias entidades distintas e agrupá-las numa única entidade genérica (superclasse). *Exemplo:* Ao analisar as entidades `Carro` , `Mota` e `Camião` , agrupam-se os atributos comuns (Matrícula, Marca) numa superclasse denominada `Veículo` .

Restrições associadas: A hierarquia pode ser definida quanto à **participação** (Total ou Parcial) e quanto à **disjunção** (Disjunta ou Sobreposta).

🔴 P23 — Transações e Propriedades ACID

Coberto nos resumos do professor

Pergunta esperada:

O que é uma transação? Descreva as propriedades ACID. Discuta as maneiras pelas quais uma transação pode ser completada.

Resposta Rápida: Uma **Transação** é uma unidade lógica de processamento que agrupa um conjunto de operações de base de dados (leituras/escritas) que devem ser executadas como um bloco único.

Para garantir a integridade dos dados face a falhas e acessos concorrentes, o SGBD deve assegurar as quatro propriedades **ACID**:

- **Atomicidade (Atomicity):** Princípio do "tudo ou nada". Todas as operações da transação são executadas com sucesso (`COMMIT`) ou revertidas (`ROLLBACK`).
- **Consistência (Consistency):** A transação deve levar a base de dados de um estado consistente para outro estado consistente, respeitando todas as restrições de integridade.
- **Isolamento (Isolation):** A execução de uma transação concorrente deve ocorrer de forma isolada das restantes.
- **Durabilidade (Durability):** Uma vez concluída a transação com sucesso (`COMMIT`), as suas alterações tornam-se permanentes.

Maneiras pelas quais uma transação pode ser completada: Uma transação é concluída de duas formas principais:

1. **COMMIT:** Quando todas as operações da transação são executadas com sucesso. O SGBD grava permanentemente as alterações na base de dados (persistência) e liberta os bloqueios de concorrência.
2. **ROLLBACK:** Quando ocorre uma falha (erro de sistema, violação de integridade ou cancelamento). O SGBD desfaz todas as alterações efetuadas desde o início da transação, devolvendo a base de dados ao seu estado inicial consistente e seguro.

🚀 P24 — Abordagens para Múltiplas Vistas de Utilizadores

Frequência: 3+ exames

Pergunta esperada:

Enuncie as abordagens para desenho de BD com múltiplas vistas de utilizadores.

Resposta Rápida: Ao projetar bases de dados corporativas de grande dimensão, as três abordagens para desenhar a base de dados a partir de visões distintas de diferentes departamentos são:

1. **Abordagem Centralizada (Integração de Requisitos):** Os requisitos de todos os utilizadores são combinados e fundidos numa única lista de requisitos global. O esquema global da BD é desenhado diretamente a partir dessa lista única.
2. **Abordagem de Integração de Vistas (View Integration):** É desenhado um esquema de base de dados local independente para cada departamento. Posteriormente, estes esquemas locais são fundidos/integrados num esquema concetual global unificado, resolvendo conflitos de nomes e estruturas.
3. **Abordagem Mista:** Combina características das anteriores. Os requisitos fundamentais e comuns a todos os departamentos são desenhados de forma centralizada de imediato, enquanto as vistas altamente específicas são tratadas como esquemas locais integrados na fase final.

🚀 P44 — Passos no Desenho Conceptual e Lógico de uma BD

Frequência: 4+ exames | **Foco Recurso:** ✅ Secundário

Pergunta esperada:

Identifique quais os principais passos/etapas a seguir na fase de Desenho Conceptual e de Desenho Lógico de uma base de dados.

Resposta Rápida: O desenho de bases de dados divide-se em duas fases principais com passos estruturados:

- **Desenho Conceptual (Criar modelo abstrato independente de SGBD):**
 1. Identificar os tipos de entidades.
 2. Identificar os tipos de relacionamentos entre entidades.
 3. Identificar e associar os atributos às entidades ou relacionamentos.
 4. Determinar os domínios de cada atributo.
 5. Determinar os atributos identificadores (chaves primárias e candidatas).
 6. Considerar conceitos de modelação avançada (especialização/generalização).
 7. Verificar a redundância no modelo.
 8. Validar o modelo conceptual com as transações dos utilizadores.
 9. Rever o modelo conceptual com os utilizadores.
- **Desenho Lógico (Mapear o modelo conceptual para o modelo relacional):**
 1. Remover componentes não compatíveis com o modelo relacional (ex: remover relacionamentos M:N introduzindo tabelas associativas, decompor atributos multivalorados).
 2. Criar as relações (tabelas) a partir do modelo conceptual.
 3. Validar as relações utilizando a técnica de **normalização** (1FN, 2FN, 3FN).
 4. Validar as relações face às transações de dados previstas.
 5. Definir restrições de integridade (chaves primárias, estrangeiras, NOT NULL, CHECK).
 6. Rever o modelo lógico com os utilizadores.

P45 — Fatores Críticos de Sucesso no Desenho de uma BD

Frequência: 3+ exames | **Foco Recurso:**  Secundário

Pergunta esperada:

Identifique quais os principais fatores críticos de sucesso no desenho de uma base de dados.

Resposta Rápida: Para garantir que o desenho de uma base de dados atenda perfeitamente aos requisitos organizacionais e seja implementado com sucesso, devem-se observar os seguintes 8 fatores críticos:

1. **Trabalho Interativo com Utilizadores:** Envolver ativamente os utilizadores finais no desenho para validar requisitos continuamente.
2. **Metodologia Estruturada:** Seguir uma metodologia formal e sistemática passo a passo.
3. **Foco na Integridade:** Incorporar desde o início as restrições de integridade nos modelos.
4. **Combinação de Técnicas:** Reunir conceptualização, normalização e validação das transações.
5. **Uso Intensivo de Diagramas:** Representar graficamente o modelo (diagramas ER).

6. **Uso de LDBD:** Utilizar a Linguagem de Desenho de Bases de Dados para expressar restrições semânticas adicionais.
7. **Dicionário de Dados:** Construir e manter um dicionário de dados detalhado.
8. **Iterabilidade:** Estar preparado para repetir passos e refinar os modelos à medida que novas informações surgem.

🚀 P46 — Ciclo de Vida de uma Aplicação de Bases de Dados

Frequência: 3+ exames | **Foco Recurso:** ☒ Secundário

Pergunta esperada:

Descreva quais os passos/estágios a desenvolver no ciclo de vida de uma aplicação de Bases de Dados.

Resposta Rápida: O desenvolvimento de uma aplicação de BD segue 11 etapas sequenciais e cíclicas:

1. **Planeamento da Base de Dados:** Planear e definir os recursos, tarefas e cronograma de desenvolvimento.
2. **Definição do Sistema:** Delimitar o âmbito, os objetivos da aplicação e identificar os utilizadores.
3. **Recolha e Análise de Requisitos:** Reunir informações sobre a organização e as necessidades dos utilizadores (*Fact-finding*).
4. **Desenho da BD:** Criar os desenhos conceptual, lógico e físico da base de dados.
5. **Seleção de SGBD (opcional):** Escolher o software de SGBD comercial mais adequado.
6. **Desenho da Aplicação:** Projetar as interfaces com o utilizador e a lógica dos programas que acedem à BD.
7. **Prototipagem (opcional):** Desenvolver uma versão preliminar para testes rápidos de validação.
8. **Implementação:** Criar fisicamente as tabelas na BD (DDL) e compilar o código das aplicações.
9. **Conversão e Alimentação:** Transferir dados de sistemas antigos para a nova BD.
10. **Testes:** Executar testes exaustivos para detetar falhas lógicas ou de desempenho.
11. **Manutenção Operacional:** Monitorizar a aplicação em produção, otimizar consultas e efetuar backups regulares.

🚀 P47 — Arquitetura de Referência para SGBDs Distribuídos

Frequência: 3+ exames | **Foco Recurso:** ☒ Secundário

Pergunta esperada:

Descreva a arquitetura de referência para SGBDs Distribuídos e compare-a com a arquitetura ANSI/SPARC.

Resposta Rápida: Um SGBD Distribuído gere uma coleção de bases de dados logicamente relacionadas, mas distribuídas geograficamente por uma rede de computadores. A sua arquitetura estende a arquitetura ANSI/SPARC de 3 níveis para suportar a transparência da distribuição, adicionando novos esquemas intermédios:

1. **Esquema Global Conceptual:** Define a estrutura lógica de toda a BD distribuída como se esta estivesse centralizada num único local.
2. **Esquema de Fragmentação:** Define como as tabelas globais são divididas em fragmentos mais pequenos (horizontal ou verticalmente).

3. **Esquema de Alocação:** Define em que nós/sites da rede os fragmentos de dados serão fisicamente guardados (incluindo possíveis replicações).
4. **Esquemas Conceptuais e Internos Locais:** São os esquemas lógicos e físicos de cada nó individual da rede, respeitando a arquitetura ANSI/SPARC local do SGBD que corre naquele site. *Diferença:* Enquanto a ANSI/SPARC foca no isolamento da visão do utilizador em relação ao armazenamento físico local, a arquitetura distribuída foca no isolamento em relação à localização física dos dados na rede.

🔴 P48 — Fragmentação em SGBDs Distribuídos (Razões e Desvantagens)

Frequência: 3+ exames | Foco Recurso: ✅ Secundário

Pergunta esperada:

O que é a fragmentação de dados num SGBD Distribuído? Descreva as principais razões para a sua utilização e quais as desvantagens associadas.

Resposta Rápida: A fragmentação é a divisão de uma tabela global em sub-tabelas menores, chamadas **fragmentos**, que podem ser distribuídas pela rede. Pode ser *Horizontal* (seleção de linhas) ou *Vertical* (projeção de colunas).

- **Razões para Utilização:**

- **Uso Localizado:** A maioria das aplicações interage apenas com uma parte dos dados (ex: o departamento de vendas do Porto precisa apenas dos dados dos clientes do Porto).
- **Eficiência e Desempenho:** Armazenar os fragmentos perto de onde são mais frequentemente acedidos reduz o tráfego de rede e melhora os tempos de resposta.
- **Paralelismo:** Permite dividir uma única transação global em subconsultas que executam de forma concorrente em múltiplos sites sobre fragmentos distintos.
- **Segurança:** Dados que não são necessários num determinado site não são lá guardados.

- **Desvantagens:**

- **Degradação de Desempenho Global:** Consultas que necessitam de cruzar múltiplos fragmentos armazenados em nós diferentes da rede exigem transferências lentas de dados.
- **Complexidade de Integridade:** Garantir restrições de integridade referencial entre fragmentos localizados em computadores diferentes é extremamente difícil e dispendioso.

🔴 P49 — Quatro Estratégias Alternativas para Alocação de Dados (SGBDD)

Frequência: 3+ exames | Foco Recurso: ✅ Secundário

Pergunta esperada:

Descreva quais as quatro estratégias alternativas para a alocação de dados num SGBD Distribuído.

Resposta Rápida: A alocação de dados define onde os fragmentos de dados são armazenados na rede. Existem 4 estratégias fundamentais:

1. **Centralizada:** Consiste em armazenar toda a base de dados e o SGBD num único computador central (site). Os utilizadores de outros sites acedem a este computador central através da rede.
2. **Particionada (ou Fragmentada):** A base de dados é dividida em fragmentos disjuntos (sem redundância) e cada fragmento é alocado ao site da rede onde é mais utilizado.
3. **Replicação Completa:** Consiste em manter uma cópia integral e idêntica de toda a base de dados em cada um dos sites da rede. Maximização da leitura, mas custo extremo de atualização.
4. **Replicação Seletiva (Mista):** Combinação das três estratégias anteriores. Alguns dados são mantidos centralizados, outros são particionados de forma única e os dados críticos com elevada taxa de leitura são replicados em sites estratégicos.

🚩 P50 — Principais Arquiteturas para SGBDs Paralelos

Frequência: 3+ exames | **Foco Recurso:** ✅ Secundário

Pergunta esperada:

Descreva quais as principais arquiteturas de hardware utilizadas nos SGBDs Paralelos.

Resposta Rápida: Os SGBDs Paralelos utilizam múltiplos processadores (CPUs) e discos de forma concorrente para acelerar o processamento de grandes volumes de dados. Dividem-se em 3 arquiteturas principais:

1. **Memória Partilhada (Shared Memory):** Múltiplos processadores partilham uma única memória RAM central comum e os mesmos dispositivos de armazenamento (discos) através de um canal de comunicação de alta velocidade.
 - *Limitação:* Baixa escalabilidade devido ao congestionamento no acesso à memória RAM comum.
2. **Disco Partilhado (Shared Disk):** Cada processador tem a sua própria memória RAM privada, mas todos os processadores partilham os mesmos discos físicos.
 - *Vantagem:* Reduz o congestionamento de memória, mas continua limitado pelo barramento de acesso aos discos.
3. **Nada Partilhado (Shared Nothing):** Cada processador possui a sua própria memória RAM privada e os seus próprios discos privados. Não há partilha física de recursos; a comunicação entre os nós é efetuada exclusivamente por rede. É a arquitetura mais escalável.

🚩 P51 — Razões para o Desenvolvimento de SGBD de Objetos

Frequência: 3+ exames | **Foco Recurso:** ✅ Secundário

Pergunta esperada:

Explique quais as razões/condicionantes que proporcionaram o desenvolvimento de SGBDs de Objetos.

Resposta Rápida: Os SGBDs de Objetos (SGBDOO) surgiram nos anos 90 para colmatar a desadequação do modelo relacional tradicional face a aplicações avançadas que lidam com dados complexos. As principais condicionantes foram:

1. **Incompatibilidade de Paradigmas (Impedance Mismatch):** A dificuldade em mapear objetos complexos de linguagens de programação (C++, Java) para tabelas bidimensionais relacionais.
2. **Representação de Dados Complexos:** Necessidade de armazenar estruturas complexas que contêm dados multimídia, geometria (SIG/Mapas), engenharia (CAD/CAM) ou ciência de forma natural.
3. **Abstração e Encapsulamento:** Permitir embutir na própria base de dados o comportamento (métodos/funções) e a ocultação de informação (*information hiding*).
4. **Identidade do Objeto (OID):** Identificação de objetos baseada em referências físicas permanentes geradas pelo sistema (OID), independente dos valores dos seus atributos.
5. **Suporte a Herança e Polimorfismo:** Possibilidade de especializar entidades e reaproveitar comportamento lógico e de dados na base de dados.

🚀 P52 — Fraquezas dos SGBD Relacionais para Aplicações Avançadas

Frequência: 3+ exames | **Foco Recurso:** ✅ Secundário

Pergunta esperada:

Explique porquê que as fraquezas do modelo relacional e dos SGBD Relacionais fazem com que estes sejam inadequados para aplicações de bases de dados avançadas.

Resposta Rápida: O modelo relacional clássico apresenta as seguintes limitações que o tornam ineficiente para aplicações avançadas (como SIG, CAD/CAM ou multimídia):

1. **Estrutura de Dados Rígida:** Possui apenas uma única estrutura lógica de representação de dados: a relação (tabela bidimensional de dados atômicos). Isto obriga à decomposição excessiva de objetos complexos do mundo real em dezenas de tabelas normalizadas.
2. **Operações Fixas:** O SQL disponibiliza um conjunto fixo de operações DML que não pode ser estendido com novas operações personalizadas.
3. **Fraco Suporte Semântico:** As restrições de integridade padrão (PK, FK, CHECK) são insuficientes para impor regras de negócio dinâmicas complexas sem o uso excessivo de triggers opacos.
4. **Dificuldade em Consultas Recursivas:** Realizar pesquisas hierárquicas (ex: listar toda a árvore de componentes de uma máquina complexa) é computacionalmente pesado e de escrita complexa em SQL padrão.
5. **Impedance Mismatch:** O overhead computacional e de desenvolvimento necessário para traduzir estruturas de objetos na aplicação para tabelas no SGBD.

🚀 P53 — Arquitetura típica de um Data Warehouse

Frequência: 3+ exames | **Foco Recurso:** ✅ Secundário

Pergunta esperada:

Represente graficamente ou descreva a arquitetura típica de um Data Warehouse, identificando as suas principais componentes.

Resposta Rápida: A arquitetura de um Data Warehouse (DW) organiza o fluxo de dados desde as fontes transacionais até ao utilizador analítico final, dividindo-se nas seguintes componentes:

1. **Sistemas de Dados Operacionais (Operational Data):** As bases de dados transacionais (OLTP) de produção, ficheiros e fontes externas.
2. **Operational Data Store (ODS):** Área de retenção temporária onde os dados operacionais são integrados e limpos preliminarmente.
3. **ETL Manager (Gestor de Carga):** Executa os processos de Extração, Transformação e Carga (ETL) dos dados para o DW.
4. **Warehouse Manager:** O motor do DW, responsável por gerir o armazenamento físico dos dados, criar índices, agregações e efetuar backups.
5. **Query Manager:** Componente que gere e otimiza a execução de consultas dos utilizadores finais.
6. **Detailed Data / Summarized Data:** Os dados históricos armazenados detalhadamente e as vistas agregadas (Lightly e Highly Summarized Data) para relatórios rápidos.
7. **Metadata (Metadados):** O catálogo que documenta as estruturas do DW, regras de ETL e permissões.
8. **End-User Access Tools:** As ferramentas usadas pelos utilizadores finais (OLAP, Data Mining, relatórios e dashboards).

🚀 P54 — Cinco Fluxos de Dados de um Data Warehouse

Frequência: 3+ exames | **Foco Recurso:** ☒ Secundário

Pergunta esperada:

Descreva as atividades associadas a cada um dos cinco principais fluxos de dados de um Data Warehouse.

Resposta Rápida: O funcionamento de um Data Warehouse baseia-se em 5 fluxos principais de dados:

1. **Inflow (Fluxo de Entrada):** Atividades associadas à extração de dados das fontes operacionais heterogéneas, limpeza de inconsistências, transformação de formatos e carregamento físico no DW.
2. **Upflow (Fluxo de Subida):** Atividades que agregam valor aos dados do DW através do cálculo de agregações, sumarizações e geração de Data Marts departamentais.
3. **Downflow (Fluxo de Descida):** Atividades associadas ao arquivamento de dados muito antigos e à criação de cópias de segurança (backups) regulares dos dados históricos.
4. **Outflow (Fluxo de Saída):** Atividades que disponibilizam os dados aos utilizadores finais através de ferramentas de BI, relatórios e consultas ad-hoc.
5. **Metaflow (Fluxo de Metadados):** Atividades transversais dedicadas à gestão, manutenção e atualização dos metadados (garantindo que as regras de ETL e estruturas de dados estejam documentadas e atualizadas).

Resumo: Probabilidade por Pergunta para o Recurso

Prioridade	Pergunta	Tema
🔥 🔥 🔥	P1	Integridade Referencial + ON DELETE/UPDATE
🔥 🔥 🔥	P2	Normalização: Objetivos + Desempenho
🔥 🔥 🔥	P3	Anomalias de Atualização
🔥 🔥 🔥	P4	Triggers (Definição + Vantagens/Desvantagens)
🔥 do recurso	P5	Sistemas BD vs Ficheiros + SGBD
🔥 do recurso	P6	Arquitetura ANSI/SPARC (Nível Conceptual)
🔥 do recurso	P7	Independência de Dados
🔥 do recurso	P8	Data Warehouses & Data Marts
🔥 do recurso	P25	Definição de Chaves (Candidata, Primária, Estrangeira)
🔥 do recurso	P26	Duas Principais Regras de Integridade
🔥 do recurso	P27	Cláusula GROUP BY vs WHERE vs HAVING
🔥 do recurso	P28	Circunstâncias para usar Desnormalização
🔥	P9	LMD Procedimentais vs Não-Procedimentais
🔥	P10	Subquery vs Junção
🔥	P11	Arquitetura Cliente-Servidor (2 vs 3)
🔥	P12	Atributos no Modelo ER
🔥	P13	Cursors SQL
🔥	P29	Papel do Gestor de BD (Administrador de BD)
🔥	P30	Funções que um SGBD deve satisfazer
🔥	P31	As Três Gerações de SGBD
🔥	P32	Função de um System Catalog
🔥	P33	Propósito e uso de Índices sobre Relações
🔥	P34	Controlo de Concorrência e a sua Importância

Prioridade	Pergunta	Tema
🔥	P35	Conceito de Database Schema e seus Três Níveis
🔥	P36	Objetivos da Arquitetura ANSI/SPARC
🔥	P37	Propriedades das Relações no Modelo Relacional
🔥	P38	Cláusulas do Comando SELECT
🔥	P39	Importância do WHERE em UPDATE e DELETE
🔥	P40	Combinação de Resultados de duas Queries
🔥	P41	Os Três Tipos de Subqueries
🔥	P42	Mecanismos de Controlo de Acesso em SQL (GRANT/REVOKE)
🔥	P43	Vantagens e Desvantagens das Vistas
★	P14	Materialização de Vistas
★	P15	Operações de Junção (5 tipos + exemplos)
★	P16	Stored Procedures vs Funções (UDF)
★	P17	Sublinguagens (DDL, DML, DCL, TCL)
★	P18	5 Operações Básicas de Álgebra Relacional (+ Divisão)
★	P19	Componentes do Ambiente SGBD
★	P20	Conceitos do Modelo Relacional
★	P21	Vistas Atualizáveis
★	P22	Especialização vs Generalização (ER)
★	P23	Transações e ACID (+ COMMIT/ROLLBACK)
★	P24	Abordagens Múltiplas Vistas
★	P44	Passos no Desenho Conceptual e Lógico de uma BD
★	P45	Fatores Críticos de Sucesso no Desenho de uma BD
★	P46	Ciclo de Vida de uma Aplicação de Bases de Dados
★	P47	Arquitetura de Referência para SGBDs Distribuídos
★	P48	Fragmentação em SGBDs Distribuídos

Prioridade	Pergunta	Tema
★	P49	Quatro Estratégias Alternativas para Alocação de Dados (SGBDD)
★	P50	Principais Arquiteturas para SGBDs Paralelos
★	P51	Razões para o Desenvolvimento de SGBD de Objetos
★	P52	Fraquezas dos SGBD Relacionais para Aplicações Avançadas
★	P53	Arquitetura típica de um Data Warehouse
★	P54	Cinco Fluxos de Dados de um Data Warehouse

💡 **Dica final:** O exercício de **normalização** (P7 no exame, vale 3 val.) e a **modelação com SQL + Álgebra Relacional** (P8, vale 5 val.) saem **SEMPRE** — mas com documentos e cenários diferentes. Pratica com os exames modelo de recurso!

📁 **Exames modelo de recurso disponíveis para praticar:**

- [Modelo 1](#) — TecnoShop + Companhia Aérea
- [Modelo 2](#) — AutoFlex Rent-a-Car + Ginásio
- [Modelo 3](#) — Grand Plaza Hotel + Reparação Eletrónica
- [Modelo 4](#) — Clínica Geral do Norte + Stock Peças